

Guía de endpoints

Usuarios, roles y permisos · Cóctel del Mar

Qué hace cada servicio, para qué sirve y cómo funciona por dentro

Este documento explica, uno por uno, los endpoints del Auth Service (inicio de sesión, usuarios, roles, permisos y bitácora). Para cada uno encontrarás: **para qué sirve** en la pantalla, **qué hace**, **cómo lo hace** internamente, un ejemplo de petición y respuesta, y los errores posibles. Todo lo descrito está implementado y cubierto por pruebas de integración contra una base PostgreSQL real.

Cómo está armado

- **Gateway (puerto 8080):** es el único punto de entrada. Valida que el token JWT sea auténtico y no haya vencido, responde el CORS del navegador y reenvía la petición al servicio correspondiente.
- **Auth Service (puerto 8081):** no es accesible desde fuera; vuelve a validar el token por seguridad y aplica los permisos de cada endpoint. Es quien guarda usuarios, roles, permisos y bitácora.
- **Base de datos:** PostgreSQL compartida. El esquema lo crea únicamente el módulo *db-migration*; el Auth Service solo lo valida al arrancar.

Conceptos que se repiten en todo el documento

Concepto	Qué significa
accessToken	Credencial corta (JWT firmado) que se manda en cada petición: <code>Authorization: Bearer ...</code> . Dura 15 minutos. Lleva dentro el id, el correo, los roles, los permisos y si debe cambiar su contraseña.
refreshToken	Credencial larga y opaca (cadena aleatoria) que solo sirve para pedir un par de tokens nuevo. Dura 8 horas y se usa una sola vez . En la base solo se guarda su huella (hash), nunca el valor.
Permiso	Una acción concreta que se puede otorgar (por ejemplo <i>manage_users</i>). Cada endpoint exige uno. Se asignan por persona.
Rol	Una plantilla de permisos por defecto (ADMIN, ALTAS, CONTABILIDAD, TESORERIA, COSTOS). Al asignar un rol se copian sus permisos al usuario; después se pueden editar sin tocar el rol.
Baja lógica	Nada se borra de la base: los registros se marcan <i>enabled = false</i> y dejan de aparecer.
Bitácora	Historial de auditoría: quién hizo qué sobre qué usuario y cuándo.

Tiempos y límites

Concepto	Valor
Vida del accessToken	15 minutos
Vida del refreshToken	8 horas
Inactividad máxima	30 minutos sin renovar el token: la sesión se cierra
Intentos fallidos de login	5; al quinto la cuenta se bloquea
Duración del bloqueo automático	2 horas (después se desbloquea sola)
Contraseña nueva	Mínimo 8 caracteres

Formato de los errores

El Auth Service responde siempre { "error": "mensaje", "code": "CODIGO" }. El Gateway, cuando rechaza por token ausente, vencido o contraseña pendiente, usa el formato estándar de Spring ({ "timestamp",

"status", "error", "path" }}, sin código ni mensaje.

Resumen de endpoints

Endpoint	Método	Permiso	Para qué sirve
/auth/login	POST	Público	Iniciar sesión
/auth/refresh	POST	Público	Renovar los tokens
/auth/logout	POST	Público	Cerrar sesión
/auth/me	GET	Sesión	Datos de la sesión actual
/auth/change-password	POST	Sesión	Cambiar la contraseña
/users	GET	manage_users	Listar y buscar usuarios
/users/{id}	GET	manage_users	Detalle de un usuario
/users	POST	manage_users	Dar de alta un usuario
/users/{id}	PATCH	manage_users	Editar datos personales y organización
/users/{id}/roles	PATCH	manage_users (+ manage_roles para roles)	Guardar roles y permisos
/users/{id}/status	PATCH	manage_users	Activar, inactivar o bloquear
/users/{id}/verify-email	POST	manage_users	Marcar el correo como verificado
/users/{id}/close-sessions	POST	manage_users	Cerrar todas las sesiones
/users/{id}/reset-password	POST	manage_users	Reiniciar la contraseña
/users/{id}	DELETE	manage_users	Dar de baja a un usuario
/users/{id}/audit-log	GET	view_audit_log	Historial de un usuario
/audit-log	GET	view_audit_log	Historial global
/roles	GET	manage_users o manage_roles	Catálogo de roles
/permissions	GET	manage_users o manage_roles	Catálogo de permisos

1. Sesión y autenticación

POST

/auth/login

Iniciar sesión

Para qué sirve

Es la pantalla de inicio de sesión. Entrega al front todo lo necesario para trabajar: los tokens y los datos del usuario (roles, permisos, subsidiarias, ubicaciones).

Qué hace

Valida el correo y la contraseña, abre una sesión y devuelve un par de tokens más el resumen del usuario. Cuenta los intentos fallidos y bloquea la cuenta al quinto.

Cómo lo hace

- Busca al usuario por correo (solo los que no están dados de baja). Si el correo no existe responde lo mismo que una contraseña mala, para no revelar qué correos existen.
- Si el usuario estaba bloqueado temporalmente y ya pasaron las 2 horas, lo desbloquea antes de seguir (y deja constancia en la bitácora).
- Si sigue bloqueado responde 423 aunque la contraseña sea correcta, indicando cuántos minutos faltan.
- Compara la contraseña contra su hash **bcrypt** (la contraseña nunca se guarda en claro).
- Si falla, suma un intento **en una transacción aparte** para que el conteo se guarde aunque el login termine con error. Al quinto intento marca la cuenta como bloqueada por 2 horas.
- Si acierta, reinicia el contador, registra la fecha de último acceso y genera los tokens: un JWT firmado (HS256, 15 min) y un refresh token aleatorio (8 h) del que solo se guarda el hash.

Petición

```
POST /auth/login
{ "username": "laura.gomez@cocteldelmar.com", "password": "Passw0rd!" }
(el campo "username" lleva el correo)
```

Respuesta

```
{ "accessToken": "eyJhbGciOi...", "refreshToken": "q8Zk...", "expiresIn": 900,
  "user": { "id": 12, "email": "laura.gomez@cocteldelmar.com", "displayName": "Laura Gomez",
    "phone": "3001234567", "status": "ACTIVE",
    "roles": ["CONTABILIDAD"], "permissions": ["view_dashboard"],
    "subsidiarias": [{"id": 1, "nombre": "..."}], "ubicaciones": [{"id": 7, "nombre": "..."}],
    "mustChangePassword": false, "lastLoginAt": "2026-10-06T19:04:03Z", "emailVerified": true } }
```

Errores

401 INVALID_CREDENTIALS · 423 ACCOUNT_LOCKED (el mensaje dice cuánto falta para poder reintentar).

Ojo: *expiresIn* está en segundos. Si *mustChangePassword* es true, el usuario debe cambiar su contraseña antes de usar la app.

POST

/auth/refresh

Renovar los tokens

Para qué sirve

Mantiene viva la sesión sin pedir la contraseña otra vez. El front lo llama antes de que venza el accessToken.

Qué hace

Cambia un refresh token vigente por un par nuevo (accessToken y refreshToken).

Cómo lo hace

- Busca el token por su hash. Si no existe: INVALID_REFRESH_TOKEN.
- **Si el token ya se había usado** (está revocado) lo trata como robo: cierra todas las sesiones del usuario y responde REFRESH_TOKEN_REUSED.

- Si pasaron las 8 h: REFRESH_TOKEN_EXPIRED. Si pasaron más de 30 minutos desde la última actividad: SESSION_IDLE_TIMEOUT (inactividad).
- Si el usuario ya está bloqueado o inactivo: USER_LOCKED_OR_INACTIVE.
- Si todo está bien emite un par nuevo (con los roles y permisos actuales de la base) y revoca el anterior.

Petición

```
POST /auth/refresh
{ "refreshToken": "q8Zk..." }
```

Respuesta

```
{ "accessToken": "...", "refreshToken": "...", "expiresIn": 900 }
```

Errores

401 con INVALID_REFRESH_TOKEN, REFRESH_TOKEN_EXPIRED, REFRESH_TOKEN_REUSED, SESSION_IDLE_TIMEOUT o USER_LOCKED_OR_INACTIVE.

Ojo: nunca lances dos renovaciones al mismo tiempo con el mismo token: la segunda se interpreta como robo y cierra todas las sesiones. Esta es también la forma en que el usuario recibe sus permisos actualizados.

POST

/auth/logout

Cerrar sesión

Para qué sirve

El botón "Cerrar sesión".

Qué hace

Invalida el refresh token de la sesión actual para que no pueda renovarse más.

Cómo lo hace

- Marca el token como revocado. Si el token no existe responde igual con éxito (es idempotente).
- No exige un accessToken vigente a propósito: así se puede cerrar sesión aunque el token ya haya vencido.

Petición

```
POST /auth/logout
{ "refreshToken": "q8Zk..." }
```

Respuesta

```
{ "success": true }
```

Ojo: el accessToken que ya se emitió sigue siendo válido hasta que venza (máximo 15 min); lo que se impide es renovarlo.

GET

/auth/me

Datos de la sesión actual

Para qué sirve

Al recargar la página, el front necesita saber quién es el usuario sin pedir la contraseña otra vez.

Qué hace

Devuelve los mismos datos del campo *user* del login.

Cómo lo hace

- Toma el id del usuario del token (nunca de un parámetro que el cliente pueda falsificar) y lee sus datos actuales.
- Es una de las tres rutas permitidas mientras la contraseña temporal no se haya cambiado.

Respuesta

```
{ "id": 12, "email": "...", "displayName": "...", "roles": [...], "permissions": [...], ... }
```

Errores

401 si el token venció o es inválido.

POST**/auth/change-password**

Cambiar la contraseña

Para qué sirve

Pantalla de cambio obligatorio de contraseña (primer ingreso o después de un reinicio) y cambio voluntario.

Qué hace

Reemplaza la contraseña del usuario autenticado y apaga la marca de contraseña pendiente.

Cómo lo hace

- Comprueba que la contraseña actual coincida con el hash guardado.
- Guarda el hash bcrypt de la nueva (mínimo 8 caracteres) y pone *mustChangePassword* en false.

Petición

```
POST /auth/change-password
{ "currentPassword": "LAURAGOMEZ2026", "newPassword": "NuevaClave2026" }
```

Respuesta

```
{ "success": true }
```

Errores

401 INVALID_CREDENTIALS (la actual no coincide) · 400 (la nueva es muy corta).

Ojo: el token que ya tiene el usuario sigue marcado como "contraseña pendiente". Hay que llamar a /auth/refresh justo después para obtener uno limpio.

2. Administración de usuarios

Todos estos endpoints exigen el permiso **manage_users**, salvo los que indican otro. El Gateway valida el token y el Auth Service vuelve a comprobar el permiso: si falta responde 403.

GET

/users

Listar y buscar usuarios

Para qué sirve

La tabla principal de "Administración de usuarios", con búsqueda, filtros y paginación.

Qué hace

Devuelve una página de usuarios. Los usuarios dados de baja no aparecen.

Cómo lo hace

- Construye la consulta con filtros opcionales: *search* (texto en el nombre completo o el correo, sin importar mayúsculas), *role* (código del rol) y *status* (active, inactive, blocked).
- Pagina en la base de datos (*page* empieza en 0, *size* por defecto 20).
- Resuelve las subsidiarias y ubicaciones de cada fila para mostrarlas con nombre.

Petición

GET /users?search=laura&role=CONTABILIDAD&status=active&page=0&size=20

Respuesta

```
{ "content": [ { "id": 12, "email": "...", "displayName": "Laura Gomez", "status": "ACTIVE",  
                "roles": ["CONTABILIDAD"], "permissions": ["view_dashboard"], ... } ],  
  "page": 0, "size": 20, "totalElements": 25, "totalPages": 2 }
```

Ojo: *status=blocked* incluye a los bloqueados por un administrador y a los bloqueados por intentos fallidos.

GET

/users/{id}

Detalle de un usuario

Para qué sirve

Precarga las pestañas Información personal, Organización y Seguridad y acceso del formulario de edición.

Qué hace

Devuelve todos los datos del usuario, incluidos los contadores de seguridad.

Cómo lo hace

- Lee al usuario y a su empleado (datos personales y organizacionales). El jefe directo llega con su nombre, no como id suelto.
- *activeSessions* cuenta los refresh tokens sin revocar y sin vencer. *failedLoginAttempts* es el contador de intentos fallidos. *lockedUntil* es el fin del bloqueo temporal (null si no está bloqueado o si el bloqueo es manual).
- *temporaryPassword* se **recalcula** con la misma regla del alta, solo mientras el usuario no haya cambiado su contraseña. Nunca se guarda en claro: por eso el administrador puede volver a consultarla.

Respuesta

```
{ "id": 12, "email": "...", "firstName": "Laura", "lastName": "Gomez", "displayName": "Laura Gomez",  
  "status": "ACTIVE", "roles": [...], "permissions": [...], "subsidiarias": [...], "ubicaciones": [...],  
  "mustChangePassword": true, "temporaryPassword": "LAURAGOMEZ2026",  
  "lastLoginAt": null, "failedLoginAttempts": 0, "activeSessions": 2, "lockedUntil": null,  
  "emailVerified": false, "createdAt": "...", "birthDate": "1990-05-01", "gender": "Femenino",  
  "address": { "city": "...", "state": "...", "postalCode": "...", "street1": "...", ... },  
  "department": "Tecnología", "area": "Sistemas", "jobTitle": "Analista",  
  "manager": { "id": 3, "displayName": "Carlos Medina" },  
  "employeeCode": "EMP-1000", "hireDate": "2026-01-10", "contractEndDate": null }
```

Errores

404 USER_NOT_FOUND (no existe o está dado de baja).

POST

/users

Dar de alta un usuario

Para qué sirve

El botón "Nuevo usuario". El administrador no escribe contraseña: el sistema genera una temporal.

Qué hace

Crea al empleado, su cuenta de acceso, su pertenencia organizacional y, si se indica, sus roles.

Cómo lo hace

- Rechaza el correo si ya lo usa otro usuario activo (un correo de un usuario dado de baja sí se puede reutilizar).
- Asignar roles al crear exige el permiso **manage_roles**; sin él responde 403.
- Valida que cada rol, subsidiaria y ubicación existan; si alguno no, rechaza **todo** sin guardar nada.
- Al asignar un rol **copia** los permisos por defecto de ese rol al usuario.
- Genera la contraseña temporal: **NOMBRE + PRIMER APELLIDO + AÑO**, en mayúsculas y sin acentos (Laura Gómez, 2026 = LAURAGOMEZ2026), la guarda con bcrypt y marca *mustChangePassword*.
- Registra la acción *created* en la bitácora.

Petición

```
POST /users
{ "firstName": "Laura", "lastName": "Gomez Perez", "email": "laura.gomez@coctel del mar.com",
  "phone": "3001234567", "supervisorId": 3, "subsidiariaIds": [1], "ubicacionIds": [7],
  "roleIds": ["CONTABILIDAD"], "status": "active" }
(obligatorios: firstName, lastName, email; status solo active o inactive)
```

Respuesta

```
201 { "id": 12, "email": "...", "displayName": "Laura Gomez Perez", "roles": ["CONTABILIDAD"],
      "permissions": ["view_dashboard"], "subsidiarias": [...], "ubicaciones": [...],
      "mustChangePassword": true, "temporaryPassword": "LAURAGOMEZ2026" }
```

Errores

409 EMAIL_ALREADY_REGISTERED · 400 INVALID_ROLE / INVALID_LOCATION / INVALID_USER_STATUS · 403 ROLE_ASSIGNMENT_FORBIDDEN.

Ojo: la contraseña temporal se muestra al crear; después se vuelve a ver en el detalle mientras siga pendiente.

PATCH

/users/{id}

Editar datos personales y organización

Para qué sirve

Guarda las pestañas Información personal y Organización en una sola llamada.

Qué hace

Actualiza solo los campos que llegan con valor; lo que no se manda no se toca.

Cómo lo hace

- Cambiar el correo lo deja sin verificar y comprueba que no lo use otro usuario.
- subsidiariaIds* (pertenencia) y *ubicacionIds* (acceso al portal): una lista **reemplaza** la actual, una lista vacía las quita todas y omitirla no toca nada. Lo que se quita no se borra: queda con *enabled* = false.
- Si un id de subsidiaria o ubicación no existe, rechaza la actualización completa (no se guarda ni el resto).
- Registra *updated* y, si cambió la organización, también *organization_updated*.

Petición

```
PATCH /users/12
{ "phone": "3009998888", "department": "Finanzas", "jobTitle": "Analista", "managerId": 3,
  "address": { "city": "Bogotá", "postalCode": "110111" },
  "subsidiariaIds": [1, 2], "ubicacionIds": [7] }
```

Respuesta

(resumen del usuario, igual al de cada fila de GET /users)

Errores

404 · 409 EMAIL_ALREADY_REGISTERED · 400 INVALID_LOCATION.

Ojo: un null explícito no borra un campo de texto: los nulos se ignoran.

PATCH

/users/{id}/roles

Guardar roles y permisos

Para qué sirve

El botón "Guardar permisos" de la pestaña Seguridad y acceso.

Qué hace

Reemplaza los roles y/o los permisos de una persona. Los permisos son independientes del rol.

Cómo lo hace

- Los dos campos son opcionales: omitido = no tocar; una lista reemplaza la actual por completo.
- La regla "al cambiar de rol se restablecen los permisos por defecto" la aplica el front (usa *defaultPermissions* de GET /roles) y envía el resultado final; el backend lo guarda tal cual.
- Cambiar roles exige el permiso **manage_roles** (se revisa el del usuario que hace el cambio).
- Valida roles y permisos antes de guardar; si alguno no existe, no se guarda nada.
- En la bitácora solo queda lo que cambió de verdad: *role_changed* y/o *permissions_changed*.

Petición

```
PATCH /users/12/roles
{ "roleIds": ["TESORERIA"], "permissions": ["view_reconciliation", "export_reports"] }
```

Respuesta

(resumen del usuario con sus roles y permisos nuevos)

Errores

400 INVALID_ROLE · 400 INVALID_PERMISSION · 403 ROLE_ASSIGNMENT_FORBIDDEN · 404.

Ojo: el usuario verá sus permisos nuevos en su próxima renovación de token (máximo 15 minutos).

PATCH

/users/{id}/status

Activar, inactivar o bloquear

Para qué sirve

La lista desplegable "Estatus" de la pantalla de administración.

Qué hace

Cambia el estado del usuario manualmente.

Cómo lo hace

- Valores: *active* (también desbloquea y borra el vencimiento del bloqueo), *inactive* y *blocked*.
- *blocked* es un bloqueo **manual**: no vence solo, hay que activarlo de nuevo. Se guarda como LOCKED.
- Registra *status_changed* en la bitácora.

Petición

```
PATCH /users/12/status
{ "status": "blocked" }
```

Respuesta

(resumen del usuario; su "status" viene como ACTIVE, INACTIVE o LOCKED)

Errores

400 INVALID_USER_STATUS · 404.

POST**/users/{id}/verify-email**

Marcar el correo como verificado

Para qué sirve

Botón para que un administrador confirme el correo a mano (no existe enlace ni token de verificación).

Qué hace

Pone *emailVerified* en true. No lleva cuerpo.

Cómo lo hace

- Registra *email_verified* en la bitácora. Si luego se cambia el correo, vuelve a quedar sin verificar.

Respuesta

```
{ "id": 12, "emailVerified": true }
```

Errores

404.

POST**/users/{id}/close-sessions**

Cerrar todas las sesiones

Para qué sirve

Botón "Cerrar sesiones": saca al usuario de todos sus dispositivos (por ejemplo si perdió un equipo).

Qué hace

Revoca todos los refresh tokens vigentes del usuario. No lo bloquea ni lo da de baja: puede volver a entrar.

Cómo lo hace

- Marca como revocados sus tokens activos y registra *sessions_closed* con el número de sesiones cerradas.

Respuesta

```
{ "id": 12, "activeSessions": 0 }
```

Errores

404.

Ojo: el accessToken ya emitido sigue valiendo hasta que venza (máx. 15 min); lo que se impide es renovarlo.

POST**/users/{id}/reset-password**

Reiniciar la contraseña

Para qué sirve

Botón "Restablecer contraseña": cuando alguien olvidó la suya o hay que recuperar el acceso.

Qué hace

Devuelve al usuario a su contraseña temporal, lo obliga a cambiarla y cierra sus sesiones.

Cómo lo hace

- Recalcula la temporal con la misma regla (NOMBRE + PRIMER APELLIDO + año de alta), guarda su hash y marca *mustChangePassword*.
- Revoca todas sus sesiones activas y registra *password_reset* en la bitácora.

Respuesta

```
{ "id": 12, "temporaryPassword": "LAURAGOMEZ2026" }
```

Errores

404.

Ojo: el administrador debe comunicarle la contraseña al usuario; el sistema no envía correos.

DELETE**/users/{id}**

Dar de baja a un usuario

Para qué sirve

Elimina a un usuario de la operación conservando su historial.

Qué hace

Baja lógica: el usuario deja de poder entrar y desaparece de listados y detalle, pero su fila y su bitácora se conservan.

Cómo lo hace

- Marca `enabled = false` y revoca sus sesiones. Registra *deleted* en la bitácora.
- Libera su correo: se puede dar de alta a otra persona con el mismo.
- Un administrador **no puede darse de baja a sí mismo**.

Respuesta

```
{ "success": true, "id": 12 }
```

Errores

404 (no existe o ya estaba dado de baja) · 409 CANNOT_DELETE_SELF.

3. Bitácora

Estos endpoints exigen el permiso **view_audit_log** (no *manage_users*): una persona puede tener uno sin el otro.

GET**/audit-log**

Historial global

Para qué sirve

El botón "Historial" de la pantalla de administración de usuarios.

Qué hace

Lista, de la más reciente a la más antigua, las acciones registradas sobre usuarios.

Cómo lo hace

- Filtros opcionales: *userId*, *action* y *search* (texto libre que busca en el detalle y en el nombre o correo de quien actuó o sobre quien se actuó, sin importar mayúsculas).
- Cada entrada trae el actor y el objetivo con su nombre. El actor es null cuando lo hizo el sistema (bloqueo o desbloqueo automático).

Petición

```
GET /audit-log?action=role_changed&search=laura&page=0&size=20
```

Respuesta

```
{ "content": [ { "id": 51, "actor": { "id": 1, "displayName": "Administrador Principal" },  
                "target": { "id": 12, "displayName": "Laura Gomez" },  
                "action": "sessions_closed", "detail": "Sesiones cerradas por administrador (2 activas)",  
                "createdAt": "2026-10-06T19:04:03Z" } ],  
  "page": 0, "size": 20, "totalElements": 1, "totalPages": 1 }
```

Errores

403 si falta `view_audit_log`.

Acciones que se registran

Acción	Cuándo se registra
created	Alta de usuario

Acción	Cuándo se registra
updated	Edición de datos (PATCH /users/{id})
organization_updated	La edición cambió subsidiarias o ubicaciones
status_changed	Cambio manual de estado
locked	Bloqueo automático por intentos fallidos (sin actor)
unlocked	Desbloqueo manual (con actor) o automático al vencer las 2 horas (sin actor)
role_changed	Cambió el rol
permissions_changed	Cambiaron los permisos
password_reset	Reinicio de contraseña
email_verified	Verificación manual del correo
sessions_closed	Cierre de sesiones
deleted	Baja lógica

GET

/users/{id}/audit-log

Historial de un usuario

Para qué sirve

La pestaña Historial dentro del detalle de un usuario.

Qué hace

Igual que el historial global pero de una sola persona. Funciona también si el usuario ya fue dado de baja.

Cómo lo hace

- Acepta *action*, *search*, *page* y *size*. Responde 404 solo si el usuario nunca existió.

Respuesta

(misma página que /audit-log)

Errores

404 USER_NOT_FOUND · 403 si falta view_audit_log.

4. Catálogos de roles y permisos

Permiten al front no escribir listas a mano. Exigen **manage_users** o **manage_roles**. Los catálogos se cargan con migraciones de base de datos; no se editan desde la pantalla.

GET

/roles

Catálogo de roles

Para qué sirve

La lista desplegable de rol en el alta y la edición.

Qué hace

Devuelve los roles con los permisos que traen por defecto, para que el front recalculé los checkboxes al cambiar de rol.

Cómo lo hace

- Lee los roles habilitados con sus permisos por defecto, ordenados.

Respuesta

```
[ { "code": "ADMIN", "name": "Administrador",  
  "defaultPermissions": ["capture_sales", "export_reports", "import_settlements", "manage_differences",  
    "manage_roles", "manage_users", "view_audit_log", "view_dashboard", "view_reconciliation"] },  
  { "code": "ALTAS", "name": "Encargado de altas de usuarios",  
    "defaultPermissions": ["manage_roles", "manage_users", "view_audit_log"] } ]
```

GET

/permissions

Catálogo de permisos

Para qué sirve

Los checkboxes de la pestaña Seguridad y acceso.

Qué hace

Devuelve todos los permisos con su descripción y el módulo bajo el cual se agrupan en pantalla.

Cómo lo hace

- El campo *module* es el título del grupo (por ejemplo "Conciliación bancaria").

Respuesta

```
[ { "code": "view_dashboard", "description": "Ver resumen de venta", "module": "Resumen de venta" },  
  { "code": "manage_users", "description": "Administrar usuarios", "module": "Administración de usuarios" } ]
```

Los 9 permisos y qué habilitan

Permiso	Módulo	Qué habilita
view_dashboard	Resumen de venta	Consultar ventas (resumen y bitácora de capturas)
capture_sales	Resumen de venta	Lanzar la captura manual de ventas
view_reconciliation	Conciliación bancaria	Ver la conciliación
manage_differences	Conciliación bancaria	Gestionar diferencias
import_settlements	Conciliación bancaria	Importar liquidaciones (CSV)
export_reports	Conciliación bancaria	Exportar reportes
manage_users	Administración de usuarios	Todo /users y los catálogos
manage_roles	Administración de usuarios	Asignar roles; ver catálogos
view_audit_log	Administración de usuarios	Ver la bitácora

Roles y sus permisos por defecto

Rol	Nombre	Permisos por defecto
ADMIN	Administrador	Los 9
ALTAS	Encargado de altas de usuarios	manage_users, manage_roles, view_audit_log
CONTABILIDAD	Contabilidad	view_dashboard
TESORERIA	Tesorería	view_reconciliation, manage_differences, import_settlements, export_reports
COSTOS	Costos	ninguno

5. Códigos de error

HTTP	Código	Cuándo
400	INVALID_ROLE / INVALID_PERMISSION	Un rol o permiso indicado no existe
400	INVALID_LOCATION	Subsidiaria o ubicación inexistente o dada de baja
400	INVALID_USER_STATUS	Estado fuera de active / inactive / blocked
401	INVALID_CREDENTIALS	Correo o contraseña incorrectos
401	INVALID_REFRESH_TOKEN / REFRESH_TOKEN_EXPIRED	Refresh token desconocido o vencido (8 h)
401	REFRESH_TOKEN_REUSED	Se reusó un refresh token: se cierran todas las sesiones
401	SESSION_IDLE_TIMEOUT	Más de 30 minutos sin renovar
401	USER_LOCKED_OR_INACTIVE	Al renovar, el usuario ya está bloqueado o inactivo
403	PASSWORD_CHANGE_REQUIRED	Debe cambiar la contraseña temporal
403	ROLE_ASSIGNMENT_FORBIDDEN	Asignar roles sin el permiso manage_roles
403	(sin código)	Falta el permiso que exige el endpoint
404	USER_NOT_FOUND	El usuario no existe o está dado de baja
409	EMAIL_ALREADY_REGISTERED	El correo ya lo usa otro usuario activo
409	CANNOT_DELETE_SELF	Un administrador intenta darse de baja a sí mismo
423	ACCOUNT_LOCKED	Cuenta bloqueada (intentos fallidos o bloqueo manual)

6. Bloqueo por intentos fallidos

- Al **quinto** intento fallido de login la cuenta se bloquea **2 horas**. Mientras dure, el login responde 423 aunque la contraseña sea correcta, y el mensaje dice cuántos minutos faltan.
- Pasadas las 2 horas el usuario se desbloquea solo: al intentar entrar, o por un barrido automático cada minuto, lo que ocurra primero. Su contador de intentos vuelve a cero y queda constancia en la bitácora.
- Un administrador puede desbloquearlo antes poniendo su estado en *active*.
- Un bloqueo **manual** (*blocked*) **no vence solo**: solo se levanta activando al usuario.
- La duración (2 horas) es provisional y se cambia en la configuración del Auth Service.